

**Artificial
Intelligence**
AI-2002

Lab 10
Supervised Learning 1

National University of Computer & Emerging Sciences – NUCES –
Karachi



National University of Computer & Emerging Sciences - NUCES -
Karachi
FAST School of Computing

Course Code: AI-2002	Artificial Intelligence Lab
-----------------------------	------------------------------------

1. Objective

2. Supervised Learning Libraries

- 2.1 Scikit Learn
- 2.2 Seaborn
- 2.3 Numpy
- 2.4 Pandas
- 2.5 Matplotlib

3. Data Preprocessing Techniques

- 3.1 Reading and Writing data
- 3.2 Handling Missing Values
- 3.3 Encoding Categorical Data
 - 3.3.1 Label Encoding (For Binary Categories)
 - 3.3.2 One-Hot Encoding (For Multi-Class Categories)
- 3.4 Data Visualization

4. Train-Test Split and Cross Validation

- 4.1 K-Fold Cross Validation
- 4.2 Leave-One-Out Cross Validation (LOOCV)

5. Performance Evaluation Metrics

- 5.1 Confusion Matrix
- 5.2 ROC Curve



1. Objective

1. The fundamentals of machine learning and its various types.
2. How supervised learning differs from other types of machine learning.
3. The usage of Python libraries like NumPy, Pandas, Matplotlib, and Scikit-learn for implementing machine learning algorithms.
4. Practical implementation of supervised learning algorithms such as Linear Regression and Decision Trees.
5. How to evaluate the performance of machine learning models using metrics like accuracy, mean squared error, and classification reports.

2. Supervised Learning Libraries

Supervised learning is a type of machine learning where the model is trained on labeled data. The key libraries for supervised learning in Python include:

2.1 Scikit Learn

Scikit-learn is a robust machine learning library that provides algorithms for classification, regression, clustering, and dimensionality reduction. It includes tools for model selection, preprocessing, and evaluation.

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)
print("Predictions:", model.predict(X))
```

2.2 Seaborn



National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

Seaborn is a visualization library built on top of Matplotlib that provides attractive and informative statistical graphics. It is widely used for:

- Creating plots like scatter plots, histograms, and box plots.
- Displaying distributions and relationships between variables.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Sample dataset
data = pd.DataFrame({'x': [1,2,3,4,5], 'y': [2,4,6,8,10]})

# Create scatter plot
sns.scatterplot(x='x', y='y', data=data)
plt.title("Scatter Plot")
plt.show()
```

2.3 Numpy

NumPy is a fundamental package for numerical computing in Python. It provides:

- Multi-dimensional arrays (ndarray).
- Mathematical operations (mean, standard deviation, linear algebra).
- Random number generation.

```
import numpy as np

# Example: Create a NumPy array and perform basic operations
arr = np.array([1, 2, 3, 4, 5])
print(np.mean(arr)) # Mean of the array
```

2.4 Pandas

Pandas is used for data manipulation and analysis. It provides:

- **DataFrames** – A tabular data structure like an Excel spreadsheet.



National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

- **Series** – One-dimensional labeled arrays.
- Functions for reading/writing CSV, Excel, SQL, JSON.

```
import pandas as pd

# Loading a CSV file into a DataFrame
df = pd.read_csv('data.csv')

# Display first 5 rows
print(df.head())
import numpy as np

# Example: Create a NumPy array and perform basic operations
arr = np.array([1, 2, 3, 4, 5])
print(np.mean(arr)) # Mean of the array
```

2.5 Matplotlib

Matplotlib is used for creating static, animated, and interactive visualizations. It provides Line plots, bar plots, scatter plots along with the customization of axes, labels, colors, and styles.

```
import matplotlib.pyplot as plt

categories = ["A", "B", "C", "D"]
values = [10, 20, 15, 30]
plt.bar(categories, values, color='green')
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Bar Chart Example")
plt.show()
```

3. Data Preprocessing Techniques

Data preprocessing is a crucial step in machine learning that involves cleaning, transforming, and organizing raw data to make it suitable for model training. It includes



National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

handling missing values, encoding categorical variables, feature scaling, normalization, and more.

3.1 Reading and Writing data

For example, consider reading a csv file and copying the data in another csv file.

```
import pandas as pd
df = pd.read_csv("data.csv") # Reading a CSV file
df.to_csv("output.csv", index=False) # Writing to a CSV file
print(df.head()) # Display first few rows
```

3.2 Handling Missing Values

Missing values can cause issues in model training. We can handle them in multiple ways:

```
# Check for missing values
print(df.isnull().sum())
# Drop missing values
df_cleaned = df.dropna()
print("Dataset after dropping missing values:", df_cleaned.shape)

# Handle missing values
df['age'] = df['age'].fillna(df['age'].mean()) # Fill age with mean
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0]) # Fill
embarked with mode

# Convert deck to string and replace NaN
df['deck'] = df['deck'].astype(str).fillna('Unknown')
# Drop remaining NaN values if needed
df = df.dropna()
print(df.isnull().sum()) # Confirm no missing values
```

3.3 Encoding Categorical Data

Machine learning models require numerical data, so we must convert categorical variables.



National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

3.3.1 Label Encoding (For Binary Categories)

```
#Converts categories into numbers (e.g., male → 0, female → 1).  
le = LabelEncoder()  
df['gender'] = le.fit_transform(df['gender']) # 0 for male, 1 for female  
print(df['gender'].head())
```

3.3.2 One-Hot Encoding (For Multi-Class Categories)

```
#Creates separate binary columns for each category.  
df = pd.get_dummies(df, columns=['embarked'], drop_first=True)  
print(df.head())
```

3.4 Data Visualization

Data visualization is the process of representing data graphically using plots, charts, and graphs. It helps to identify patterns and trends in the data.

```
#creating histogram of Numerical Features  
df[['age', 'fare']].hist(bins=20, figsize=(8,4))  
plt.show()
```

4. Train-Test Split and Cross Validation

The train-test split is a technique used in machine learning to evaluate the performance of a model. It involves splitting the dataset into two subsets:

1. **Training Set:** Used to train the model.
2. **Testing Set:** Used to evaluate the model's performance on unseen data.

This helps in assessing how well the model generalizes to new data and prevents issues like overfitting.

Underfitting vs Overfitting

- **Underfitting:** Model is too simple, performs poorly on training and test data.



National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

- **Overfitting:** Model memorizes training data but performs poorly on unseen data.
- **Solution:** Use train-test split and cross-validation to check generalization.

Different Ways to Split Data

1. **80-20 Split (Common Practice)**
 - 80% Training, 20% Testing
 - Used for general ML problems.
2. **70-30 Split**
 - More test data, useful when we need better validation.
3. **60-20-20 Split (Train, Validation, Test)**
 - 60% Training, 20% Validation, 20% Testing
 - Used for hyperparameter tuning to avoid overfitting.

To understand train-test split, let's use it with the **Naïve Bayes Classifier**:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
import numpy as np

# Generate dummy dataset
X = np.random.rand(100, 5) # 100 samples, 5 features
y = np.random.randint(0, 2, 100) # Binary target variable (0 or 1)

# Splitting the data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Create a Naïve Bayes model
model = GaussianNB()

# Train the model on training data
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
```




```
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")
```

4.1 K-Fold Cross Validation

Instead of using a single train-test split, K-Fold Cross-Validation divides the dataset into **K** equal parts (**folds**). The model is trained **K** times, each time using **K-1** folds for training and **1** fold for validation. The average performance across all **K** folds is used as the final evaluation metric.

```
from sklearn.model_selection import KFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
import seaborn as sns

# Load dataset
df = sns.load_dataset("titanic")

# Select features and target, handling missing values
X = df[['age', 'fare']].fillna(df[['age', 'fare']].mean())
y = df['survived']

# Convert to DataFrame to use .iloc[]
X = pd.DataFrame(X)
y = pd.Series(y)

# Define K-Fold (5 splits)
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Initialize model
model = LogisticRegression()

# Initialize model
```



```
model = LogisticRegression()

# Store accuracy scores
accuracy_scores = []

# Perform K-Fold CV
for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index] # Now X is a
    DataFrame
    y_train, y_test = y.iloc[train_index], y.iloc[test_index] # Now y is a
    Series

    # Train model
    model.fit(X_train, y_train)

    # Predict and evaluate
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracy_scores.append(acc)

# Print average accuracy
print("K-Fold CV Average Accuracy:", np.mean(accuracy_scores))
```

4.2 Leave-One-Out Cross Validation (LOOCV)

Leave-One-Out Cross-Validation (**LOOCV**) is an extreme case of K-Fold where K = total number of samples. The model is trained on all data except one sample, and the process repeats for each sample.

```
from sklearn.model_selection import LeaveOneOut

# Initialize LOOCV
loo = LeaveOneOut()

# Store accuracy scores
loo_scores = []

# Perform LOOCV
```



```
-----  
for train_index, test_index in loo.split(X):  
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]  
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]  
  
    # Train model  
    model.fit(X_train, y_train)  
  
    # Predict and evaluate  
    y_pred = model.predict(X_test)  
    loo_scores.append(accuracy_score(y_test, y_pred))  
  
# Print average accuracy  
print("LOOCV Average Accuracy:", np.mean(loo_scores))  
-----
```

5. Performance Evaluation Metrics

Evaluation metrics in machine learning are used to assess the performance of a model or algorithm by quantifying how well it is doing its intended task. The choice of evaluation metrics depends on the specific problem you are trying to solve and the nature of your data. Here are some commonly used evaluation metrics in machine learning:

- Accuracy
- Precision
- Recall
- R2 Score
- ROC AUC Curve

These are just a few of the many evaluation metrics used in machine learning. The choice of metric depends on the problem at hand and the goals of your analysis. Selecting

the most appropriate metric based on your specific use case is essential to accurately assess the model's performance.

Evaluation Metrics are different for different tasks. The Evaluating parameters are different for classification and Regression. Creating an AI based ML or DL model will be beneficial but it would be great to measure the performance of ML or DL models. For measuring the performance of the model different metrics have been used for different purposes. Here are the main Evaluation Metrics used for Classification and Regression.



National University of Computer & Emerging Sciences - NUCES -
Karachi
FAST School of Computing

Classification	Regression
Accuracy Score	Mean Absolute Error
F1 Score	Mean Square Error
Precision	Max Error
Recall	R2 Score
ROC_AUC Score	Explained Variance Score
Balanced Accuracy Score	D2 Absolute Score

5.1 Confusion Matrix

Classification Models have multiple categorical outputs. Most error measures will calculate the total error in our model, but we cannot find individual instances of errors in our model. The model might misclassify some categories more than others, but we cannot see this using a standard accuracy measure.

Furthermore, suppose there is a significant class imbalance in the given data. In that case, i.e., a class has more instances of data than the other classes, a model might predict the majority class for all cases and have a high accuracy score; when it is not predicting the minority classes. This is where confusion matrices are useful.

A confusion matrix presents a table layout of the different outcomes of the prediction and results of a classification problem and helps visualize its outcomes. It plots a table of all the predicted and actual values of a classifier.

How to Create a 2x2 Confusion Matrix?

We can obtain four different combinations from the predicted and actual values of a classifier:



National University of Computer & Emerging Sciences - NUCES -
Karachi
FAST School of Computing

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

- **True Positive:** The number of times our actual positive values are equal to the predicted positive. You predicted a positive value, and it is correct.
- **False Positive:** The number of times our model wrongly predicts negative values as positives. You predicted a negative value, and it is actually positive.
- **True Negative:** The number of times our actual negative values are equal to predicted negative values. You predicted a negative value, and it is actually negative.
- **False Negative:** The number of times our model wrongly predicts negative values as positives. You predicted a negative value, and it is actually positive.

5.2 ROC Curve

ROC stands for Receiver Operating Characteristic, which is a commonly used tool for evaluating the performance of binary classification algorithms. An ROC curve is a plot of the true positive rate (TPR) against the false positive rate (FPR) for a binary classifier at different classification thresholds. The TPR is the proportion of true positives (correctly

predicted positive cases) out of all actual positives, while the FPR is the proportion of false positives (incorrectly predicted positive cases) out of all actual Negatives.

An ideal classifier will have a TPR of 1 and an FPR of 0, which would be represented by a point in the top left corner of the plot. A completely random classifier would have a diagonal line from the bottom left to the top right corner of the plot, with an AUC of 0.5. A good classifier will have a curve that is closer to the top left corner, indicating a higher TPR and a lower FPR.

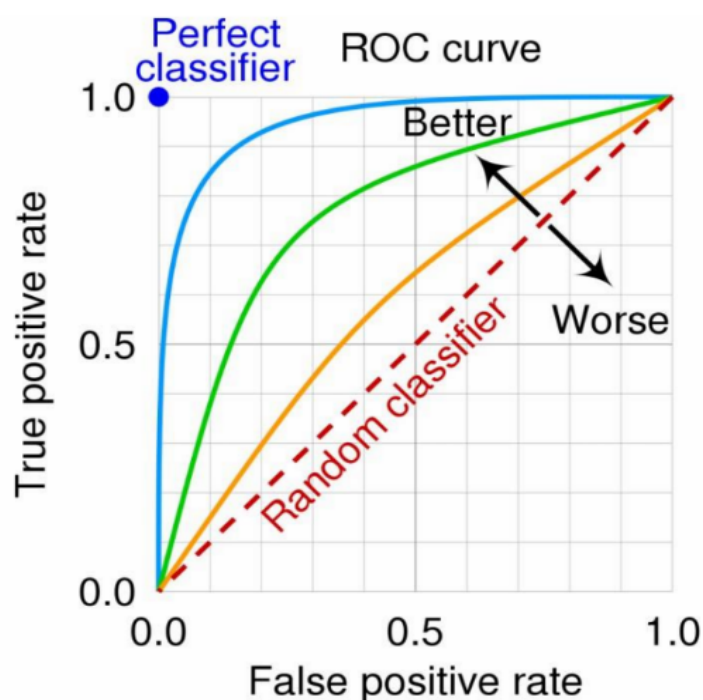


National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

For example, if we had a binary classifier that was predicting whether an email was spam or not, and we plotted its ROC curve, it might look like a curve that rises steeply at the beginning (high TPR and low FPR), then levels off as the threshold increases, eventually approaching the

diagonal line as the threshold approaches 1 (low TPR and high FPR). The area under the curve (AUC) would give us an overall measure of the classifier's performance, with higher values indicating better performance.



```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# Assuming the model has been trained and 'PredictionDT' holds class
# predictions
# To compute ROC, we need the probability estimates, not just the predicted
# class.
# For binary classification, use the probabilities of the positive class.
```



National University of Computer & Emerging Sciences - NUCES - Karachi

FAST School of Computing

```
# Get probabilities for the positive class
probabilities = DT.predict_proba(x_test)[:, 1] # Get the probability for class
'1'

# Calculate ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, probabilities)

# Calculate ROC_AUC Score
roc_auc = roc_auc_score(y_test, probabilities)

# Plot ROC curve with shaded area under the curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC Curve (AUC =
{roc_auc:.2f})')
plt.fill_between(fpr, tpr, color='skyblue', alpha=0.4)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve with AUC Area')
plt.legend(loc='lower right')
plt.show()
```
